

Assignment II

Robust Reinforcement Learning under Stochastic Action Failure

Total Marks = 15 Marks

Intended Learning Outcome:

Students should be able to

- Understand the basic functionality of Q-learning, DQN, and DDQN Methods.
- Implement the concepts of Q-learning, DQN, and DDQN.

Prerequisite:

- Students should go through the lectures CS7 - CS12;
- Webinar demonstrations

Please note that this assignment will involve some amount of self-learning (on the part of modelling solutions appropriately + programming skills)

Submission Deadline: 07th August, 2026

Instructions:

- Read the assignment proposal carefully.
- Solve all the assignment problems. Submit only one solution file. (Team # - Q_learning_DQN_DDQN)
- It is mandatory to submit the assignment in **PDF format**, consisting of all the codes and outputs with each and every iteration. Any other format will not be accepted.
- Only one submission per group is required.
- Add comments and descriptions to every function you are creating or operation you are performing. If not found, then 1 mark will be deducted. There are many assignments that need to be evaluated. Providing the comments and descriptions will help the evaluator to understand your code quickly and clearly.
- **Group Contribution Declaration:** At the beginning of the report, list the names of all group members and specify the percentage contribution of each member.
- No last moment submissions /email submissions will be accepted. Therefore make sure you will submit the first version of your assignment 2 days before the deadline.
- Assignment must be executed in virtual lab and screenshots with timestamp in virtual lab must be attached in the final submission
- Plagiarized submissions will not be evaluated at both the parties.
- Key factors which will be considered for scoring includes (but not limited to) : Creativity, Originality, Uniqueness, Code Modularity, Representation, Reproducibility, Similarity to AI tool solutions.

How to reach out for any clarifications:

This assignment is administered by

(1) Subash - subasharun@wilp.bits-pilani.ac.in

(2) Divya K - divyak@wilp.bits-pilani.ac.in

(3) Dincy R Arikkat - dincyrarikkat@wilp.bits-pilani.ac.in

Any request for clarification must be addressed through email (official email only) to all the three instructors listed.

Messaging in TEAMS is discouraged. This is to ensure we maintain track of all the transactions. If we find any clarifications to be shared across all the students, we will share this using discussion forums.

Question:

The **LunarLander-v3** environment in Gymnasium consists of:

- **State Space:** An 8-dimensional continuous state vector representing the lander's position, velocity, orientation, angular velocity, and leg-contact information.
- **Action Space:**

Action	Description
0	Do nothing
1	Fire left orientation engine
2	Fire main engine
3	Fire right orientation engine

In practical robotic systems, control commands may fail because of hardware faults, communication delays, or actuator unreliability. In this assignment, investigate the effect of **stochastic actuator failures** on reinforcement learning by creating a modified version of the LunarLander-v3 environment and comparing the performance of **Deep Q-Network (DQN)** and **Double Deep Q-Network (DDQN)** agents.

Create a custom environment by wrapping the original **LunarLander-v3** environment using **gym.Wrapper**. The wrapper must modify the **action execution process** and the **reward function** while keeping all other aspects of the original environment unchanged.

For every call to **step(action)**, the wrapper must perform the following operations in the given order.

Step 1: Receive the Agent's Action

Let a denote the action selected by the agent from the action space defined above. Store this action before applying any modifications. The stored action will later be used to compute the modified reward.

Step 2: Simulate Intermittent Engine Failure

Before passing the action to the original environment, modify the selected action according to the following rules.

- If $a = 0$, execute the action without modification.
- If $a \in \{1, 2, 3\}$:
 - Generate a random number r from a uniform distribution in the interval $[0, 1)$.
 - If $r < 0.15$, replace the selected action with **0 (Do Nothing)**.
 - Otherwise, execute the originally selected action.

Let a_{exec} denote the action that is finally executed.

The agent **must not receive any indication** that its selected action was replaced.

Example

Agent Action (a)	Random Number	Executed Action (a_{exec})
0	—	0
1	0.08	0
2	0.63	2
3	0.12	0
3	0.87	3

Step 3: Execute the Action

- Execute a_{exec} in the original LunarLander-v3 environment.
- Receive the following outputs from the base environment:

- observation
- base reward (R_{base})
- terminated
- truncated
- info

Step 4: Compute the Modified Reward

Replace the reward returned by the original environment with where

- is the reward returned by the original environment.
- is the **action originally selected by the agent** (before engine-failure simulation).
- is an indicator function that equals **1** when the selected action is **1**, **2**, or **3**, and **0** otherwise.
- is the landing bonus defined as

Note

The fuel penalty depends on the **action selected by the agent**, **not** on the action actually executed.

Therefore,

- if the agent selects a thruster action and the engine fires successfully, subtract **0.3**;
- if the agent selects a thruster action but the engine misfires and is replaced by **Do Nothing**, subtract **0.3**;
- if the agent selects **Do Nothing**, no penalty is applied.

Step 5: Safe Landing Bonus

Award the landing bonus only if all of the following conditions are satisfied simultaneously:

- `terminated == True`
- `truncated == False`
- The final observation satisfies:
 - `observation[6] == 1` (left landing leg in contact with the landing pad)
 - `observation[7] == 1` (right landing leg in contact with the landing pad)
 - `|observation[2]| < 0.10` (horizontal velocity)
 - `|observation[3]| < 0.10` (vertical velocity)
 - `|observation[4]| < 0.10` radians (orientation angle)

The relevant elements of the observation vector are:

Observation Index	Description
<code>observation[2]</code>	Horizontal velocity
<code>observation[3]</code>	Vertical velocity

observation[4]	Orientation angle
observation[6]	Left landing leg contact (0 = No, 1 = Yes)
observation[7]	Right landing leg contact (0 = No, 1 = Yes)

The landing bonus must be added **only if all of the above conditions are satisfied simultaneously**. If any condition is not satisfied, including crashes, unstable landings, episode truncation, or incomplete leg contact, **no landing bonus shall be awarded**. If any of the above conditions is not satisfied, including crashes, unstable landings, episode truncation, or incomplete leg contact, set .

Step 6: Return the Environment Output

Return the following values to the agent:

- observation
- modified reward (R)
- terminated
- truncated
- info

No additional information regarding engine failures or reward modifications should be included in the returned info dictionary.

Components That Must Remain Unchanged

Except for the modifications described above,

- the observation space must remain unchanged;
- the action space must remain unchanged;
- the state-transition dynamics of the original environment must remain unchanged (except for the stochastic action replacement described above);
- the episode termination and truncation conditions must remain unchanged; and
- no additional observations or state variables may be introduced.

Tasks

(a) Modified Environment Implementation and Verification (2.5 Marks)

Implement the modified LunarLander-v3 environment exactly as specified in the **Environment Modifications** section.

Verify the correctness of your implementation by executing multiple episodes using a random policy. Demonstrate experimentally that:

- Approximately **15%** of all attempted thruster actions are replaced by **Do Nothing** actions.
- The fuel penalty is applied for **every attempted thruster action**, irrespective of whether the engine fires successfully or misfires.
- The **+50** landing bonus is awarded **only** when the **specified safe landing criterion** is satisfied.

Present appropriate statistics, logs, or experimental evidence to support your verification.

(b) Deep Q-Network (DQN) (4 Marks)

Implement and train a **Deep Q-Network (DQN)** agent consisting of:

- Q-network
- Experience Replay Buffer
- ϵ -greedy exploration strategy
- Target Network

Train the agent on:

- the **modified** LunarLander-v3 environment; and
- the **original** LunarLander-v3 environment,

using the **same random seed, neural network architecture, optimizer, hyperparameters, and training duration** for both experiments.

c) Double Deep Q-Network (DDQN) (4 Marks)

- Implement and train a **Double Deep Q-Network (DDQN)** agent.
 - Use the **same network architecture, optimizer, replay buffer, exploration strategy, random seed, hyperparameters, and training duration** as the DQN implementation.
 - The **only difference** between the DQN and DDQN implementations should be the computation of the target Q-value.
- Train the DDQN agent on both the **modified** and **original** environments under identical experimental conditions.

(d) Performance Evaluation (2 Marks)

Compare the following four trained agents:

- DQN – Original Environment
- DDQN – Original Environment
- DQN – Modified Environment
- DDQN – Modified Environment

Plot and analyse the following:

- **Episode reward versus training episode.**
- **Average predicted Q-value versus training episode**, where the Q-value is computed using a **fixed validation set of states** that remains unchanged throughout training.
- **Successful landing rate versus training episode**, computed using a moving average over the previous 100 episodes.
- **Average number of thruster activations per episode versus training episode.**

Provide a brief interpretation of each plot, highlighting the differences between DQN and DDQN and between the original and modified environments.

(e) Discussion (2.5 Marks)

Based on your experimental results, answer the following questions.

- Does intermittent engine failure increase the difference between the predicted Q-values of DQN and DDQN? Justify your answer using the Q-value plots.
- Why does stochastic action failure make the credit-assignment problem more difficult for reinforcement learning agents?
- Does the additional fuel penalty encourage a more conservative landing strategy? Support

your answer using experimental evidence.

- Which algorithm performs better under stochastic engine failures? Is this behaviour consistent with the theoretical advantage of DDQN over DQN? Explain.
- Identify one limitation of your experimental setup and suggest one possible improvement.